# Hierarchical Hybrid Planning for Multi-AGV Scheduling in Multi-Floor Factories

[AUTHORS]
[AFFILIATION]

[EMAIL]

Abstract—Multi-floor manufacturing requires automated guided vehicles (AGVs) to share both floor routes and lift resources. We extend the prioritized planning baseline of Guo et al. (ICARCV 2024) with a hybrid architecture: a central server assigns jobs, splits routes at lifts, and books lift trips, while one server per floor plans local route legs. Lift-lobby waiting points let delays be absorbed near floor-level conflicts instead of at the factory's single parking node. In the tested three-floor layout (27 nodes, 4 lifts), ten job sets of 100 jobs and 20 AGVs, the hybrid configuration reduces mean last-delivery makespan by 65%, total tardiness by 72%, and load imbalance by 80% relative to the paper-style centralized configuration, with busiest-server CPU down 92%. All reported plans pass an independent verifier. These are experimental results for the tested layout and settings, not a general optimality or scalability guarantee.

Index Terms—Automated guided vehicles, multi-agent path finding, prioritized planning, multi-floor logistics, lift coordination, hierarchical scheduling.

## I. Introduction

Automated guided vehicles (AGVs) are a central part of material handling in smart manufacturing, moving workpieces between storage, processing, and delivery locations. Coordinating a fleet is a joint assignment, routing, and traffic-management problem: vehicles may need the same narrow aisle, workstation approach, or transport resource at overlapping times. A schedule must assign jobs and produce routes and timings that avoid collisions while meeting operational targets such as completion time, delivery due dates, and balanced vehicle utilization.

These requirements become more demanding in multi-floor facilities, where lifts connect otherwise separate floor networks. A lift is a shared resource with finite capacity and non-negligible travel time: a conflict at one lift can delay routes on several floors and propagate through the entire schedule. The prioritized planning baseline of Guo et al. [1] assigns transport tasks and plans routes sequentially by AGV priority, resolving conflicts against previously planned higher-priority vehicles. In that baseline, however, AGVs may wait only at the factory's single parking node. With lifts, the next available floor leg may be blocked by traffic on another floor, and parking-node-only waiting can serialize delays that would be better absorbed near the lift itself.

This paper extends the prioritized-planning baseline with a hybrid architecture for a multi-floor factory. A central server performs job assignment, divides each AGV route into floor-level legs at lift edges, and books lift trips, which remain shared across floors and therefore require system-wide coordination. One edge server per floor plans local route legs using prioritized conflict-free routing, and lift-lobby waiting points let a floor server delay a leg at its entry lobby instead of returning the AGV to parking. Job assignment, lift booking, dispatch, merging, and verification stay centrally orchestrated; the floor servers do not negotiate with each other directly.

The implementation is evaluated against the paper-style centralized configuration (waiting at parking only) and a centralized variant with the same lift lobbies, on ten job sets of 100 jobs and 20 AGVs in a tested three-floor layout with 27 nodes and 4 lifts. Every generated plan passes an independent verifier. The hybrid configuration reduces mean last-delivery makespan by 65%, total tardiness by 72%, and load imbalance by 80% relative to the paper-style centralized configuration; its busiest-server planning CPU time falls by 92%. These figures describe the tested factory, parameters, and seeds, not general guarantees for all multi-floor factories.

The three narrow contributions are: (i) a corrected reimplementation of the ICARCV 2024 prioritized-planning baseline with independent full-plan verification; (ii) a hybrid central-plus-floor-server architecture that separates central job assignment and lift booking from per-floor route-leg planning; and (iii) lift-lobby waiting points that absorb delays near floor-level conflicts and lift resources.

The remainder is organized as follows. Section II reviews directly related work; Section III defines the scheduling model; Section IV describes the algorithms and the hybrid extension; Section V reports the setup and results; Section VI concludes and states limitations.

## II. Related Work

Guo et al.'s ICARCV 2024 method is the baseline for this study: large-scale multiple-AGV scheduling as prioritized assignment and route planning, resolving conflicts against previously planned higher-priority vehicles [1]. The sequential approach avoids joint search over all vehicle paths, but its result depends on assignment and priority order.

Prioritized wait adjustment starts from individual paths and shifts waiting using safe intervals; its preliminary evaluation reports 50–250 agents and trades solution quality for runtime [2]. It is our competing planner (WA) in Section V.

Conflict-based search (CBS) offers a coupled alternative. Continuous-time CBS methods model nonuniform action durations and geometric collisions [3]; He et al. extend CBS to multi-floor MAPF with single-agent elevator occupancy and reset intervals, explicitly including lift conflicts in search [4]. These methods provide stronger conflict reasoning, while our experiments focus on job assignment, prioritized route windows, and lift arbitration in a decomposed planner.

Optimization-based heuristics scale to larger fleets: Hu et al. use reservation timetables with variable-neighborhood search on 3–200 AGVs [8], and Verma et al. allow multiple reservations at shared resource points [9]. At very high scale, MAPF-LNS2 repairs colliding subsets of agents and has solved warehouse instances with up to 8,000 agents [10], though its warehouse experiments do not model lifts as a constrained resource.

Hierarchical and zone-based planning motivates layered coordination: the spare-zone method of Zhao et al. uses a hierarchical allocation layer and local coordination for vehicles following fixed paths [5]; Pratissoli et al. use a centralized, decoupled, multilayer traffic manager with topological sectors [6]; HMAPP combines high-level region sequences with regional conflict-based searches [7]. The present implementation instead partitions local route windows by floor while retaining central job assignment and lift booking.

## III. Problem Model

The model uses the prioritized assignment and route-window notation of the ICARCV baseline [1], with explicit lift and lobby elements for a multi-floor factory.

### A. Sets and graph

Let $G = (N, E)$ be the undirected factory graph. Each undirected edge represents a bidirectional track. Let $F$ be the set of floors, $N_f \subseteq N$ the nodes on floor $f \in F$, and $L \subseteq E$ the set of lift links between floors. The parking node is $\pi \in N$. Let $J$ be the set of jobs and $V$ the set of AGVs; each job is served by exactly one AGV. Let $M_i$ index the successive node visits on AGV $i$'s route, including repeated visits to the same physical node.

### B. Parameters

Each job $j \in J$ has pickup and delivery nodes $p_j, p'_j \in N$ ($p_j \neq p'_j$) and desired times $t_j, t'_j$. Service times are $\eta$ (loading) and $\eta'$ (unloading); safety separation is $\delta > 0$; AGV speed is $v > 0$. Each edge $e \in E$ has length $\ell_e$; for lift edges, $\ell_e$ is the configured lift length. Travel time along an edge is $\tau_{uv} = \ell_{\{u,v\}}/v$, and $d(u, w)$ is shortest-path travel time. Jobs are considered in increasing desired pickup time $(t_j, j)$; AGV $i$ has fixed priority rank $\rho_i$ (smaller means higher priority). Assignment uses fitness weight $\lambda \geq 0$ and Algorithm 3 is limited to $k_{\max}$ passes.

### C. Decision and schedule variables

The assignment variable is $x_{ji} \in \{0, 1\}$, where $x_{ji} = 1$ means AGV $i$ is assigned job $j$: $\sum_{i \in V} x_{ji} = 1 \; \forall j \in J$. For each AGV $i$, $R_i = (n_{i,0}, \ldots, n_{i,q_i})$ is its ordered route with $n_{i,0} = n_{i,q_i} = \pi$, constructed by concatenating shortest paths through each assigned job's pickup and delivery nodes and back to $\pi$. For visit $m$, $lt_{im}$ and $rt_{im}$ are the start and end of its node-occupancy window; dwell equals $\eta$ at pickup, $\eta'$ at delivery, and zero at pass-through visits. The edge window for consecutive visits is $[rt_{i,m-1}, lt_{im}]$ with fixed duration $\tau_{n_{i,m-1}, n_{i,m}}$: the AGV does not stop while traversing an edge.

### D. Assignment quantities

When job $j$ is considered, with AGV $i$'s current node $s_i$ and available time $vt_i$:

$$\begin{aligned} ct_{ji} &= vt_i + d(s_i, p_j) + \eta + d(p_j, p'_j) + \eta', \\ et_{ji} &= \max\big(0,\ t_j - \big(vt_i + d(s_i, p_j) + \eta\big)\big), \\ dt_{ji} &= \max(0,\ ct_{ji} - t'_j), \\ F_{ji} &= ct_{ji} + \lambda(et_{ji} + dt_{ji}). \end{aligned}$$

The job is assigned to the AGV minimizing $F_{ji}$ (ties by priority); then $vt_i$ and $s_i$ are updated to $ct_{ji}$ and $p'_j$.

### E. Conflict separation

Let $W_n$ be fixed occupancy windows of higher-priority AGVs at ordinary node $n$, and $W'_{uv}$ fixed edge windows on directed traversal $u \to v$ (a physical track is exclusive in both directions). Overlapping use of an ordinary node or the same bidirectional track by distinct AGVs must be separated by at least $\delta$: for intervals $[a, b]$ and $[c, d]$, require $c \geq b + \delta$ or $a \geq d + \delta$. The parking node is exempt from node separation. Algorithm 3 resolves a conflict by shifting a route prefix right by cumulative delay $st^*$ (sweep backward from the conflict visit to the nearest previous exempt node, limited to $k_{\max}$ passes), extending the preceding exempt node's window by $st^*$.

### F. Objectives

Let $C_j$ be job $j$'s actual delivery-completion time and $U_i$ AGV $i$'s usage time (return time to $\pi$ when serving jobs). Then

$$\begin{aligned} M &= \max_{j \in J} C_j && \text{(makespan, last delivery)}, \\ T &= \sum_{j \in J} \max(0, C_j - t'_j) && \text{(total delivery tardiness)}, \\ I &= \max_{i \in V} U_i - \min_{i \in V} U_i && \text{(load imbalance)}. \end{aligned}$$

Possible objectives are $\min \; \alpha M + \beta T + \gamma I$ or $\text{lexmin}(M, T, I)$; the code does not optimize either combined objective, it reports $M, T, I$ as metrics.

**Algorithm 1** Prioritized job assignment (Algorithm 1)

Require: Jobs $J$, AGVs $V$, fitness weight $\lambda$, service times $\eta, \eta'$
Ensure: Assignment $x_{ji}$ and complete routes $R_i$

for each $j \in J$ in order $(t_j, j)$ do
  for each $i \in V$ do
    Compute $ct_{ji}, et_{ji}, dt_{ji}$ from AGV state $(s_i, vt_i)$.
    $F_{ji} \leftarrow ct_{ji} + \lambda(et_{ji} + dt_{ji})$.
  end for
  Choose $i^* \leftarrow \arg\min_i(F_{ji}, \rho_i)$; set $x_{ji^*} \leftarrow 1$.
  Append shortest paths $s_{i^*} \to p_j \to p'_j$ with dwell $\eta, \eta'$.
  Update AGV state to $(p'_j, ct_{ji^*})$.
end for
for each $i \in V$ do
  Append a shortest path from the final assigned station to $\pi$.
end for

**Algorithm 2** Prioritized conflict-free routing (Algorithm 2)

Require: Routes $R_i$, dwell times, priority order $V_1, \ldots, V_{|V|}$
Ensure: Timed plans and frozen reservations $W_n, W'_e$

Initialize empty node and edge reservation sets $W_n, W'_e$.
for each AGV $i$ in priority order do
  Construct node and edge windows along $R_i$.
  for each candidate window conflicting with $W_n$ or $W'_e$ do
    Apply Algorithm 3 to delay the route segment.
  end for
  Freeze ordinary node windows in $W_n$ and track windows in $W'_e$.
end for

**Algorithm 3** Sliding-window conflict elimination (Algorithm 3)

Require: Conflict visit $p$, reservations $W$, route $R$, limit $k_{\max}$
Ensure: Conflict-free shifted windows and delay $st^*$

Find the nearest preceding exempt visit $q$.
for $k = 1$ to $k_{\max}$ do
  Set $st^* \leftarrow 0$ and $mt^* \leftarrow \infty$.
  for visits $m = p, p-1, \ldots, q+1$ do
    Evaluate $f(lt_m + st^*, rt_m + st^*, W)$; update $mt^*$ and $st^*$.
    if $st^* > mt^*$ then
      Stop this backward sweep.
    end if
  end for
  if the sweep completes within $mt^*$ then
    break
  end if
end for
if no feasible sweep completed then
  Apply the fallback delay at $q$ until blocking reservations clear.
end if
Shift windows $q+1, \ldots, p$ and extend the window at $q$ by $st^*$.

## IV. Method

The method combines the prioritized assignment and route-window procedures of the centralized baseline [1] with a hierarchical floor decomposition. A central server assigns jobs, splits routes at lifts, books lift trips, and merges plans; one edge server per floor plans local route legs.

Figure 1 shows the information flow. Jobs and fleet state go to the central server; legs and reservations cross between central and floor servers; merged, verified schedules go to the AGVs for execution.

### A. Algorithms 1–3 recap

Algorithm 1 assigns jobs in order $(t_j, j)$ to the AGV minimizing $F_{ji}$. Algorithm 2 processes AGVs in priority order $V_1 \to V_{|V|}$, constructing node and edge windows; a window conflicting with frozen reservations triggers Algorithm 3, which delays the affected route segment. Algorithm 3 sweeps backward from the conflict visit to the nearest preceding exempt node, accumulating the smallest right-shift that clears higher-priority windows with separation $\delta$, up to $k_{\max}$ passes; on exhaustion it delays departure at the exempt node until blocking traffic clears.

### B. Hybrid central-plus-floor-server extension

The hybrid planner cuts each complete route at lift edges into floor-local legs. Each floor server schedules its legs using local node and track reservations; when needed, the AGV waits at its entry lobby or at $\pi$. The central server reserves the lift track at the earliest available time after the AGV is ready, respecting safety separation, then releases the next floor leg at the arrival lobby. Floor and lift windows are merged into complete plans before independent verification.

Legs are dispatched either in priority order or first-come-first-served (FCFS): FCFS orders legs by release time, AGV priority, then leg index; priority order processes higher-priority AGVs' legs first and is used as a decomposition check. On the tested seeds, priority-order hybrid plans match the centralized-with-lobbies plans. Configuration C (the default hybrid) uses FCFS order.

## V. Experiments

### A. Setup

Table I summarizes the layout, job model, planner parameters, and configurations. All configurations use $\eta = \eta' = 16$ s, $\delta = 10$ s, $\lambda = 1.0$, and $v = 1$ m/s; lift links use their configured 40 m length (40 s non-stop travel). The main comparison uses 100 jobs, 20 AGVs, seeds 1–10, and the strict track rule; the fleet sweep uses 4, 8, 12, 16, and 20 AGVs over the same job-generation settings. Every generated schedule is passed to an independent verifier that checks route endpoints, node and bidirectional-track separation, edge travel durations, dwell windows, and exactly-once job service; a run is accepted only when the verifier returns no violations.

TABLE I
Experimental setup (layout, job model, parameters, configurations).

| | |
|---|---|
| Layout | 3 floors; 27 operational nodes (9 per floor); 38 edges; 4 lifts (40 m each); parking $\pi = 1$A; 26 non-parking, non-lobby job stations |
| Jobs | pickup and delivery sampled uniformly from the 26 stations; $t_j \sim U[0, 600]$ s; slack $s_j \sim U[30, 200]$ s; $t'_j = t_j + d(p_j, p'_j) + \eta + \eta' + s_j$ |
| Fleet | 20 AGVs (AGV-01–AGV-20), fixed priorities $V_1$–$V_{20}$; fleet sweep: 4, 8, 12, 16, 20 AGVs |
| Parameters | $\eta = \eta' = 16$ s; $\delta = 10$ s; $\lambda = 1.0$; $v = 1$ m/s; lift travel 40 s |
| Main experiment | 100 jobs, 20 AGVs, seeds 1–10, strict track rule |
| Configurations | A: centralized paper-style (wait at parking only); B: centralized with lobbies; B2: centralized with lobbies and FCFS leg order; C: hybrid central + floor servers, lobbies, FCFS leg order |

**Algorithm 4** Hybrid floor-leg planning and lift booking

Require: Assigned routes, floor planners, central lift calendar, dispatch order
Ensure: Merged complete AGV plans
1: Split each route into maximal floor-local legs at every lift.
2: Insert each initial leg into the dispatch queue using the chosen order.
3: while the queue is nonempty do
4: Remove the next leg and schedule it with its floor planner.
5: if the route continues on another floor then
6: Reserve the intervening lift trip centrally.
7: Release the next leg at the arrival lobby after lift travel.
8: end if
9: end while
10: Merge floor-leg and lift windows; independently verify each plan.

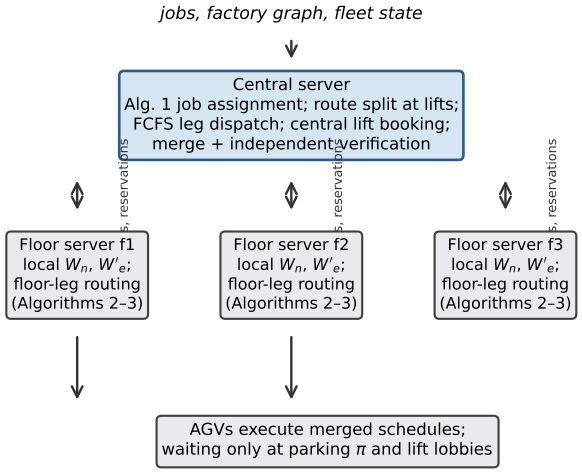



Fig. 1. Hybrid architecture: central job assignment, route split, lift booking, and verification; per-floor route-leg planning; AGVs execute merged schedules, waiting only at parking $\pi$ and lift lobbies.

## B. Main results: Gantt chart as main visual evidence

Figure 2 shows the A and C plans for seed 7 (100 jobs, 20 AGVs) on a shared time axis; rows are AGVs in priority order, colors are activities (moving on floor 1/2/3, lift trip, loading $\eta$, unloading $\eta'$), and hatched bars are waiting. Panel titles give data-derived totals. What to notice: under A the rows form a staircase of waiting that descends with priority. The annotation marks AGV-20: it waits 6,737.3 s before its first move under A but only 375.0 s under C, while its service time is 160.0 s in both configurations — the same AGV's workload is unchanged, the waiting almost entirely disappears. Figure 3 explains the hybrid: for AGV-03 (10 lift trips, floors 1–3), the route is cut into floor legs joined by lift trips, and the waiting between legs sits at lift lobbies rather than on floor nodes.

Table II reports the ten-seed means with sample standard deviations. C reduces makespan by 65%, tardiness by 72%, and imbalance by 80% relative to A. B2 and C have identical schedule outcomes on every seed, so the B2–C contrast isolates the floor-server split's computation/state distribution (server CPU 7.6 vs 22.9 ms at the busiest server; 337.2 central–floor messages) rather than a schedule-quality change.

Planning CPU is wall-clock dependent. Across five repeated runs of the main experiment, the busiest-server reduction stays between 92% and 93% (medians 98.6 ms for A and 7.4 ms for C).

## C. Fleet-size sweep

Tardiness decreases with fleet size for A at every tested step. For B it decreases through 16 AGVs, then rises from 117,682 s at 16 AGVs to 122,279 s at 20. For B2 and C it decreases slightly from 90,343 s to 90,104 s over that final step. Imbalance falls for A and rises for B, B2, and C over this sweep. Tardiness does not fall monotonically in all configurations; these are observations for the tested layout and seeds, not a general trend.

## D. Ablation: lobbies versus FCFS leg order versus floor split

The A–B contrast isolates the lobby effect (same centralized planner; B adds exempt lift-lobby waiting nodes). The B–B2 contrast isolates FCFS leg order (same centralized planner and lobbies; B2 processes legs by release time). The B2–C contrast isolates the floor-server split: schedule metrics match on every seed, so distributing computation across floor servers changes reservation state and planning distribution, not schedule quality, when the leg order is held fixed.

## E. Competing planner

The prioritized wait-adjustment planner (WA) computes individual paths and inserts waits to avoid conflicts using the same seeded jobs, layout, fleet, speeds, and safety gap. In the 10-seed comparison every WA plan passed verification; mean makespan 5,999.9 s, tardiness 226,879.9 s, and imbalance 5,144.8 s versus 2,768.9, 90,104.5, and 1,441.2 s for C. On these tested cases WA




Fig. 2. How to read: each row is one AGV in priority order (V01 highest priority); colors are activities (moving on floor 1/2/3, lift trip, loading $\eta$, unloading $\eta'$); hatched bars are waiting. Panels show the seed-7 job set under A (top) and C (bottom) on a shared time axis; dashed vertical lines mark each panel's last delivery (the bars continue to the final return to parking). Under A the rows form a staircase of waiting that descends with priority. The annotation marks AGV-20: it waits 6,737.3 s before its first move under A but only 375.0 s under C, while its service time is 160.0 s in both configurations. Panel titles give data-derived totals: last delivery 7,547.5 versus 2,549.8 s, load imbalance 6,734.6 versus 1,503.8 s, total waiting 62,169.9 versus 26,293.1 s.

did not outperform the hybrid; this is not evidence that it cannot on other instances.

## VI. Conclusion

This paper presented a hybrid architecture for prioritized multi-AGV scheduling in a multi-floor smart-manufacturing layout: a central server assigns jobs, splits routes into floor legs at lifts, and books shared lift trips; floor servers plan local route-leg schedules; lift-lobby waiting points absorb delays near floor-level conflicts and lifts; an independent checker validates every complete plan. On the tested three-floor layout (27 nodes, 4 lifts), ten job sets of 100 jobs and 20 AGVs, the hybrid configuration reduced mean makespan by 65%, tardiness by 72%, and imbalance by 80% relative to the paper-style centralized configuration, with busiest-server CPU down 92%. B2 and C matched schedule outcomes on every seed, so measured gains come from lobby waiting and FCFS leg ordering; the floor decomposition changes where reservation state and computation sit.

The reduction in tardiness is substantial, but mean total tardiness remains 90,104.5 s per 100-job seed for C. Conflict resolution and decomposition alone do not ensure deliveries meet their targets: assignment is greedy and priority-dependent, and delay accumulated through shared routes, lift use, and safety separation can still push deliveries past desired times. Reducing residual tardiness requires changes to assignment, sequencing, and resource coordination beyond distributing the planning workload. These results apply to the tested layout, fleet sizes, seeds, and parameters; they establish neither optimality nor a general scalability claim.

a) Future work.: Dynamic job arrivals, battery limits and charging, multiple lifts per floor with explicit capacity, decentralized lift coordination, and comparisons with auction-based assignment, DRL, and CBS on common instances with shared verification rules.

## Appendix A
## Reproducibility

The main experiment is regenerated by running, from agv_code/agv_code/:

python agv_compare.py --seeds 1-10 --jobs 100 --agvs 20

TABLE II
Mean ± sample standard deviation over ten seeds (100 jobs, 20 AGVs). Percentages compare C with A. B2 and C have identical schedule outcomes on every seed, so B2 is omitted from the columns.

| Metric | A | B | C | C vs. A |
|---|---|---|---|---|
| Last-delivery makespan (s) | $7,972.4 \pm 819.1$ | $3,705.9 \pm 309.8$ | $2,768.9 \pm 195.8$ | −65% |
| Total tardiness (s) | $316,893.6 \pm 41,316.3$ | $122,278.8 \pm 11,430.7$ | $90,104.5 \pm 11,447.3$ | −72% |
| Load imbalance (s) | $7,149.9 \pm 772.6$ | $2,832.3 \pm 291.4$ | $1,441.2 \pm 241.1$ | −80% |
| Total waiting (s) | $64,689.7 \pm 8,466.2$ | $27,647.7 \pm 2,747.6$ | $27,104.5 \pm 2,180.2$ | −58% |
| Longest AGV wait (s) | $7,133.4 \pm 817.9$ | $2,830.8 \pm 302.2$ | $1,887.7 \pm 173.9$ | −74% |
| Busiest-server CPU (ms) | $92.5 \pm 23.0$ | $19.9 \pm 8.7$ | $7.6 \pm 1.2$ | −92% |
| Windows on busiest server | $2,030.7 \pm 84.5$ | $1,733.5 \pm 69.5$ | $634.5 \pm 30.0$ | −69% |

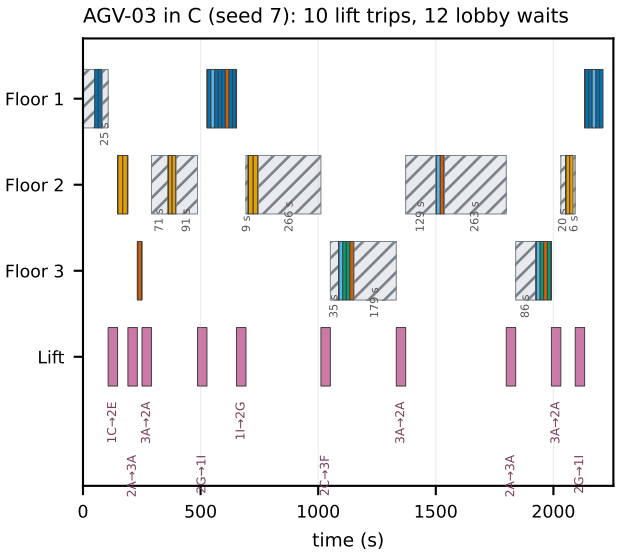



Fig. 3. How the hybrid builds one route, shown for AGV-03 on the same seed. Rows are the three floors and the lifts; the AGV's route is cut into floor legs joined by 10 lift trips (labelled with their lobby-to-lobby nodes). Hatched bars at lift-entry lobbies are lobby waits: 12 waits totalling 1,179.5 s. Between floor legs the AGV waits at lift lobbies rather than holding a floor node.

It writes to results_hybrid_100j_20a_s1-10_strict/; the folder name encodes its settings, so reruns with different settings cannot overwrite earlier results. The CPU-repeat runs use --sweep '' --outdir results_cpu_repeat_r1 (and r2–r5). Every result folder is documented in results_manifest.json (command, git commit hash, seeds, jobs, AGVs, track rule, date, Python and package versions, CPU model). test_results_manifest.py fails if a folder's jobs/AGVs/seeds/track rule disagree with its manifest entry or its folder name. make_paper_numbers.py reads the manifest-listed result files and writes paper/generated/numbers.tex and the table bodies; check_paper_numbers.py fails if a result numeral in the paper is hard-coded instead of a macro from numbers.tex or a generated table body. Machine: Intel Core i5-10210U @ 1.60 GHz, Python 3.12.3, NumPy 2.5.4, NetworkX 3.7, Matplotlib 3.11.2, Pillow 13.0.0, SciPy 1.18.1 (WSL2). These records are from the manifest at the time the numbers were produced; runtime measurements are machine- and load-dependent.

## References


[1] Y. Guo, J. Yao, J. Li, R. Su, and K. V. Ling, "Prioritized planning for large-scale multiple-AGV scheduling problem in smart manufacturing," Proc. ICARCV 2024, pp. 19–24, 2024.

[2] A. Andreychuk and K. Yakovlev, "Multi-robot path planning in well-formed infrastructures: prioritized planning vs. prioritized wait adjustment (preliminary results)," arXiv:1807.01909, 2018.

[3] A. Andreychuk, K. Yakovlev, D. Atzmon, and R. Stern, "Multi-agent pathfinding with continuous time," Artificial Intelligence, 2022.

[4] H. He, X. Wu, S. Zhao, and Z. Ren, "Conflict-based search for multi-agent path finding with elevators," arXiv:2602.20512, 2026.

[5] H. Zhao et al., "Spare-zone-based hierarchical multi-AGV path planning," Proc. ICRA, 2021.

[6] F. Pratissoli, R. Brugioni, N. Battilani, and L. Sabattini, "Hierarchical traffic management of multi-AGV systems with deadlock prevention applied to industrial environments," IEEE Trans. Automation Science and Engineering, vol. 21, no. 3, pp. 3155–3169, 2024.

[7] H. Zhang et al., "A hierarchical approach to multi-agent path finding," Proc. SoCS, vol. 12, no. 1, pp. 209–211, 2021.

[8] Y. Hu, H. Yang, and Y. Huang, "Conflict-free scheduling of large-scale multi-load AGVs in material transportation network," Transportation Research Part E, vol. 158, p. 102623, 2022.

[9] P. Verma, J. M. Olm, and R. Suárez, "Traffic management of multi-AGV systems by improved dynamic resource reservation," IEEE Access, vol. 12, pp. 19790–19805, 2024.

[10] J. Li, Z. Chen, D. Harabor, P. J. Stuckey, and S. Koenig, "MAPF-LNS2: Fast repairing for multi-agent path finding via large neighborhood search," Proc. AAAI Conf. AI, vol. 36, no. 9, pp. 10256–10265, 2022.

[11] Soma1111111, "decentralized-Autonomous-guided-vehicles," https://github.com/Soma1111111/decentralized-Autonomous-guided-vehicles.